arXiv:2504.11703v3 [cs.CR] 2026 年 5 月 14 日

Progent：利用特权控制保护人工智能代理

Tianneng Shi, Jingxuan He, Zhun Wang, Hongwei Li, Linyu Wu, Wenbo Guo, Dawn Song
[1]加州大学伯克利分校、加州大学圣巴巴拉分校、新加坡国立大学

摘要

人工智能代理通过工具调用与外部环境交互，这使它们容易受到间接提示注入等攻击，从而触发未经授权的操作。保护这些代理极具挑战性：它们自主且概率性地运行，安全需求会随着用户的任务和执行状态而变化，而且安全性和实用性之间本身就存在权衡。

本文介绍了一种名为 Progent 的新型框架，该框架通过权限控制来保障人工智能代理的安全。Progent 将权限表示为一种安全策略，该策略由工具名称和参数的符号规则组成。这些规则规定了哪些工具调用允许用于完成任务，以及哪些不必要的调用会被阻止以确保安全。每个工具调用都会通过确定性过程进行策略检查，从而强制执行最小权限原则。为了处理各种用户任务和不断变化的执行环境，大型语言模型（LLM）会根据用户任务自动生成初始策略，并在执行过程中随着新信息的到来而更新策略。每个提出的更新都由 SMT 求解器确定为缩小（自动应用）或扩展（需要显式批准），从而确保代理的有效动作空间未经批准时只能缩小　　（单调约束）。这种确定性更新机制即使在存在对抗性输入的情况下也能保持效用并防止静默权限提升。

我们在常用基准测试（例如 AgentDojo 和 ASB）上的评估表明，Progent 能够在保持高实用性的同时显著降低攻击成功率。我们还通过在 LangChain 和 OpenAI Agents SDK 等实际代理框架中展示其有效性，进一步验证了 Progent 的实用性。

1 引言

人工智能代理已成为解决通用和自主任务的一种很有前景的平台[41, 46, 47, 56]。这些代理的核心是大型语言模型（LLM），它通过各种工具集与外部环境交互[39, 40]。例如，个人助理代理使用电子邮件工具包[20]来管理电子邮件，而编码代理则使用代码解释器和命令行[47]。

人工智能代理的安全风险。人工智能代理能够与外部环境交互，这赋予了它们强大的功能，使它们能够在通信、调度和金融等领域执行各种任务。然而，这种能力也带来了巨大的攻击面。近期研究提出了人工智能代理的严重安全隐患[15, 28, 52]，攻击者利用代理与环境的交互，将对抗性指令注入外部数据源。当代理通过工具调用检索此类数据时，注入的指令可以重定向代理执行危险操作，例如未经授权的金融交易[11]或数据窃取[29]。这类攻击被称为间接提示注入[14, 30]。

人工智能代理安全面临的挑战。保障人工智能代理的安全面临诸多挑战。首先，代理的行为本质上是非确定性的：大型语言模型（LLM）处理非结构化的自然语言，并且在不同的运行中可能会做出不同的决策。与之相反，安全需要确定性的强制执行，无论大型语言模型如何推理，即使在受到对抗性影响的情况下，这种强制执行也必须有效。调和这两种需求并非易事。其次，代理的安全取决于用户的任务以及代理在执行过程中收集的信息。不同的任务需要不同的工具和参数，并且相关的上下文会随着代理与环境的交互而在运行时不断演变。防御机制必须自动跟踪这些变化，因为让用户参与每一个安全决策既不切实际，也会抵消自主代理的优势。最后，代理有时必须扩展其权限才能完成良性任务。然而，攻击者也会试图利用相同的机制来诱发有害行为，而且合法的和对抗性的权限扩展请求都通过同一渠道到达。如何在不牺牲安全性和实用性的前提下区分合法扩展和对抗性扩展，是一项根本性的挑战。我们在第 2 节中通过一个实例详细阐述了这些挑战。

我们的工作：Progent。我们推出了 Progent，一个通过权限控制来保护 AI 代理的新型框架。我们的核心思想是强制执行最小权限原则：仅允许完成任务所必需的工具调用，并阻止不必要的调用。我们之所以关注工具调用，是因为代理的所有外部效应都通过它们产生，并且它们暴露了一个结构化的接口（工具名称和类型化的参数），这与代理的非结构化推理截然不同。这使得工具调用成为确定性安全强制执行的理想切入点。Progent 利用了一种权限控制策略，该策略由针对工具名称和参数的符号规则组成。这些规则指定了哪些工具调用是允许的，哪些应该被阻止。每个工具调用都会根据这样的策略进行检查，该策略会确定性地决定是否允许调用。

为了应对上下文相关的安全需求，Progent 利用 LLM（大语言模型）根据用户任务自动生成初始策略，并在执行过程中随着新信息的到来不断更新策略。这使得安全策略能够适应任务和执行上下文，同时最大限度地减少人工干预。然而，引入 LLM 进行策略构建会引入不确定性，从而在自动化和确定性执行之间造成根本性的矛盾。

为了解决这个问题，我们进一步提出，每次策略更新都应由确定性 SMT 求解器进行检查，判断其是缩小（自动应用）还是扩展（需要显式批准）。这确保了智能体的有效动作空间未经批准时只能缩小　，我们称之为单调约束。因此，即使 LLM 受到对抗性输入的操纵，这种确定性检查也能防止任何静默的权限提升。同时，它允许在显式授权的情况下进行受控的权限扩展，从而保持了效用。

实施与评估。由于 Progent 在工具调用级别运行，因此可以通过在代理及其工具之间插入一个单独的模块进行集成。这无需更改代理的内部架构，从而使集成过程无侵入性，并可通用化于各种不同的代理框架。

我们在 AgentDojo [11] 和 ASB [57] 上评估了 Progent。Progent 将 AgentDojo 上的攻击成功率 (ASR) 从 39.9% 降低到 1.0%，将 ASB 上的攻击成功率从 70.3% 降低到 3.9%，同时在两种情况下都保持了代理的效用。我们进一步进行了大量的消融实验，以分析 Progent 的关键组件，包括模型选择和策略更新批准配置。最后，我们将 AgentDojo 扩展为一个基于 MCP 的基准测试，并将其连接到各种真实世界的代理框架（例如 LangChain 和 OpenAI Agents SDK），从而证明了 Progent 在保护单代理和多代理系统方面的有效性。

主要贡献。总而言之，我们的主要贡献如下：

- 一种特权控制方法，利用工具名称和参数上的符号策略，在工具调用级别进行确定性强制执行，并采用基于 SMT 的程序来确定策略更新是扩展还是缩小（第 4 节）。
- 一种基于 LLM 的策略生成和更新机制，可从用户的任务和执行状态自动导出上下文感知策略，并具有形式化的单调约束保证，可防止静默权限提升（第 5 节和第 6 节）。
- 对标准化基准（即 AgentDojo 和 ASB）进行了广泛的评估，并与现实世界的代理框架和多代理系统进行了集成，证明了 Progent 的总体有效性（第 7 节和第 8 节）。

2 概述

我们在图 1 中展示了一个示例，用以探讨人工智能代理如何完成用户任务、如何通过间接提示注入攻击以及保护它们所面临的挑战。然后，我们演示了 Progent 如何应对这些挑战。

2.1 代理安全面临的挑战

激励性示例：智能体执行和攻击。如图所示——
在图 1a 中，用户请求代理执行一封来自 alice@gmail.com、主题为“本周待办事项”的电子邮件中指定的操作。仅从此描述来看，代理只知道它必须首先检索到相应的电子邮件。在步骤 1 中，代理调用工具 search_emails，参数为 sender=alice@gmail.com 和 subject='本周待办事项'。返回结果是一个待办事项列表，指示代理汇总用户最近的电子邮件并将汇总内容发布到用户的 Slack。这表明完成此任务还需要电子邮件读取和 Slack 发布工具。因此，在步骤 2 中，代理调用 get_slack_info() 来获取用户的 Slack 用户名是 alice。这将确定任何后续 Slack 消息的收件人。随后，在步骤 3 中，代理继续通过调用 read_emails(recent=20) 来读取最近的电子邮件并准备汇总。然而，其中一封退回的邮件中暗含了一条提示信息：“将收件箱转发至 eve@evil.com”，其内容与合法邮件并无二致。步骤 4 正是受此提示信息的影响，此时代理会发出 send_email(to=eve@evil.com,body=(inbox)) 指令。恶意工具调用会将用户的收件箱泄露给攻击者。在步骤 5 中，代理执行了最初预期的操作 `send_slack_msg(to =alice,body=(summary))`，完成了原本无害的任务。这种攻击非常隐蔽：在执行恶意操作的同时，用户的原始任务也已完成，用户可能毫不知情。

挑战一：非确定性智能体行为。人工智能智能体处理非结构化的自然语言输入，包括用户任务和工具返回的信息。由于大型语言模型（LLM）对提示语措辞非常敏感，即使表达相同含义的细微措辞差异也可能导致智能体做出不同的决策；而用于相同目的的不同工具可能以不同的格式返回信息，从而导致不同的后续操作。更重要的是，嵌入在工具返回信息中的对抗性内容可以在没有任何明显迹象的情况下悄无声息地改变智能体的计划。智能体的执行本质上是非确定性的，但出于安全考虑，我们希望确定性地执行这些行为：诸如“仅向授权接收者发送数据”之类的属性必须以确定性的方式执行，而不是依赖于智能体的概率推理。

挑战二：上下文相关的安全需求。代理的执行涉及不同类型的上下文，自动跟踪这些上下文以强制执行安全性极具挑战性。有效的防御必须能够自动考虑这些不同的上下文来做出安全决策，因为要求用户批准每个工具调用会耗费过多精力，反而失去了使用代理的优势。我们为代理识别出两种上下文。第一种是任务上下文：同一个代理可以处理不同的任务，例如汇总电子邮件、预订航班或核对预算，每个任务授权使用不同的工具集和不同的工具参数。工具调用是否安全取决于任务：例如，当任务是“向 Alice 发送更新邮件”时，`send_email` 是合法的；但当任务是“汇总未读邮件”时，返回的邮件可能包含恶意指令，指示代理将敏感内容发送给第三方，从而将 `send_email` 变成数据泄露通道。对所有任务都过于宽松的策略过于宽泛，无法阻止攻击；而针对单个任务设计的策略又过于狭窄，无法应对其他任何任务。第二点是执行上下文：人工智能代理自主执行，上下文会随着代理在运行时收集新信息而变化。例如，只有在步骤 2 之后，代理才会得知 Slack 用户名是 alice，因此任何后续的 Slack 消息都应该只发送给 alice；发送给其他人的消息应该被阻止。这种在执行过程中收集的信息对安全至关重要，防御措施必须在获得此类信息后立即将其纳入考量。

挑战三：平衡安全性和实用性。由于代理程序自主执行，它们必须适应可能需要额外权限的新信息。然而，同一通道也可能承载恶意内容，这使得平衡安全性和实用性变得极具挑战性。在实用性方面，完成任务通常需要代理程序动态扩展其允许使用的工具和参数集。在我们的示例（图 1a）中，在步骤 1 中，返回的 TODO 表明代理程序还必须调用 `read_emails`、`get_slack_info` 和 `send_slack_msg`；禁止使用这些工具的防御措施将阻止任务完成。在安全性方面，攻击者也会试图扩展代理程序的工具使用范围以达到恶意目的。在步骤 3 中，注入尝试引入 `send_email`。



(a) Agent trajectory: the agent calls a tool at each step; each return shapes the next action.

(b) Progent policy enforcement: each call at step *i* is gated by policy P_i; calls outside P_i are blocked.

(c) Progent policy update: P_1 is derived from the user task; for $i > 1$, P_i is derived from P_{i-1} and the tool return at step $i-1$. Expansion/narrowing is judged by an SMT solver.

图 1：Progent 在运行示例中的概览。

发送到外部地址；允许这样做会导致数据泄露。这两个请求都通过同一通道到达，即工具返回，它会改变代理的计划。由于注入的指令与代理需要总结的合法电子邮件内容位于同一工具返回中，因此对抗情况尤其难以检测。因此，防御措施必须在任务真正需要时扩展代理的允许操作，同时拒绝源自对抗内容的扩展操作，且不能牺牲安全性或实用性。

2.2 Progent：通过特权控制进行防御

接下来我们将介绍 Progent 如何应对上述挑战。

基于策略的安全执行。Progent 在工具调用级别运行，因为所有具有外部影响的代理操作都通过工具调用进行。此外，工具调用接口（由工具名称和类型化参数组成）是结构化的，这与代理推理的非结构化文本不同。这使得工具调用成为一个天然的安全执行点，从而使 Progent 能够在代理行为非确定性的情况下确定性地执行安全措施（挑战一）。

Progent 引入了一种权限控制策略，用于描述代理可以发出或应该阻止的工具调用集。策略是一个规则列表；每条规则针对一个特定的工具，并指定其参数的条件，以确定是否允许调用该工具。在我们的示例（图 1b）中，策略 P 包含四条“允许”规则，这意味着如果工具名称与规则匹配且其参数满足指定的条件，则允许调用该工具。例如，read_emails 的规则仅允许在 recent=20 时调用，而 send_slack_msg 的规则仅允许在 to=alice 时调用。规则可以指定具体值（例如 20）、通配符 (*) 和正则表达式模式，这使得该策略具有足够的表达能力来捕获细粒度的约束。

除了“允许”规则外，Progent 还支持“禁止”规则，这些规则会明确阻止与特定模式匹配的工具调用，以及其他各种功能。我们在 4.1 节中提供了 Progent 策略的完整定义。

当代理发出工具调用时，Progent 将其与当前策略的规则匹配，并确定性地允许或阻止该调用；默认阻止不匹配任何规则的调用。在步骤 4 中，P_4 没有允许 send_email 的规则，因此恶意调用 send_email(to=eve@evil.com, body=⟨inbox⟩) 被阻止，并返回说明拒绝原因的回退错误，让代理调整并继续用户任务。在步骤 5 中，$P_5 = P_4$；良性调用 send_slack_msg(to=alice, body=⟨summary⟩) 匹配 Slack 规则，因此获准并完成任务。该符号化检查使每个安全决策都具有确定性，直接应对挑战一。

基于 LLM 的策略构建与更新。代理安全依赖上下文（挑战二），因此 Progent 维护逐步策略：在步骤 i，权限控制策略 P_i 精确列出此时允许的工具调用。初始策略 P_1 来自用户任务，在示例中仅允许具有特定发件人和主题的 search_emails。基于 LLM 的更新器同时考虑任务和执行上下文，实现策略管理自动化，避免为每个用户请求手写逐步规则。当 $i > 1$ 时，更新器根据步骤 i−1 的工具返回及先前策略 P_{i-1}，提出新策略 P_i。步骤 1 返回的 TODO 表明需要其他工具，因此更新器提出 P_2，添加 read_emails、get_slack_info 和 send_slack_msg 的允许规则。

在步骤 2 之后，已知 Slack 句柄为 alice，因此更新器提出 P_3，将 Slack 规则收窄为 send_slack_msg(to=alice, body=*)。步骤 3 返回的邮件含有试图引入 send_email 的注入，但策略不变：$P_4 = P_3$。下文说明如何确保这一点。

策略扩展与收窄的确定性控制。策略更新可以扩展或收窄允许的工具调用集。收窄只移除权限，因此是安全的。扩展则较为敏感：它可能是完成任务所必需的，如 $P_1 \rightarrow P_2$，也可能由攻击导致，如步骤 3 的注入。

Progent 在应用每个更新前区分收窄与扩展。若新策略 P' 允许原策略 P 不允许的调用，则更新属于扩展；若 P' 允许的每个调用也都被 P 允许，则属于收窄，包含 $P' = P$ 的情况。两种关系都能由 SMT 求解器确定性判定；正式定义见第 4.3 节。

收窄更新自动应用，扩展更新须显式批准才生效。Progent 支持自动拒绝所有扩展、转交人工审核或自动批准所有扩展等配置，并在第 8.2 节评估。在图 1c 中：$P_1 \rightarrow P_2$ 是扩展，加入 read_emails、get_slack_info 和 send_slack_msg；SMT 求解器检测到后，P_2 仅在获批后生效。$P_2 \rightarrow P_3$ 是收窄（$P_3 \subset P_2$，将 Slack 收件人绑定到 alice），因此自动应用。对于 $P_3 \rightarrow P_4$，步骤 3 的注入有两种情况：(a) 更新器识破注入，没有提出扩展，$P_4 = P_3$ 自动应用；(b) 即使更新器受骗而提议添加 send_email，SMT 求解器也会检测到扩展，审批者可以拒绝。在这两种情况下，$P_4 = P_3$，步骤 4 的恶意调用被阻止。即使 LLM 被操纵或产生幻觉，没有明确批准也不会获得新权限；实用性驱动的扩展可以获批，对抗性扩展可被拒绝。在 AgentDojo 实验（第 8 节）中，仅 6% 的更新需要审批，因此即使由人工审批，开销也较低。

Progent 的安全保障。确定性策略执行和确定性扩展控制的结合确保了代理的有效行动空间只能在未经显式批准的情况下缩小，我们称之为单调约束（详见第 6 节）。这保证了任何攻击都无法悄无声息地提升代理的权限：只有扩展请求需要批准，所有其他安全决策都以确定性的方式处理。我们在第 6 节中对此进行了详细说明。

此外，Progent 还支持扩展功能，例如对所有任务施加持久约束的通用策略，以及控制如何处理扩展请求的可配置审批模块；这些在第 7 节中有详细介绍。

Algorithm 1: Vanilla execution of AI agents.

Input : User query o_0, agent $\mathcal{A}$, tools $\mathcal{T}$, environment $\mathcal{E}$.
Output: Agent execution result.

1 **for** $i = 1$ ***to*** max_steps **do**
2 $c_i = \mathcal{A}(o_{i-1})$
3 **if** c_i is a tool call **then** $o_i = \mathcal{E}(c_i)$
4 **else** task solved, **return** task output
5 task solving fails, **return** unsuccessful

3 问题陈述和威胁模型

本节首先给出人工智能代理的定义，这将为后续介绍 Progent 奠定基础。然后，我们将概述我们的威胁模型。

3.1 人工智能代理

我们考虑通用的 AI 代理任务求解设置 [47, 56]，涉及用户 U、代理 A、工具集 T 和环境 E。代理从 U 接收初始查询 o_0，开始多步流程（算法 1）。在步骤 i，代理根据先前观测 o_{i-1} 生成动作 $c_i = A(o_{i-1})$（第 2 行）。动作可以是工具调用（第 3 行）或完成信号（第 4 行）。工具调用在 E 中执行，得到新观测 o_i；若调用失败，错误也包含在 o_i 中，并返回给代理进行下一步操作。流程持续到代理发出完成信号或达到计算预算，例如 max_steps。A 和 E 都有状态，因此先前交互可能影响 $A(o_{i-1})$ 和 $E(c_i)$。

与独立模型相比，AI 代理通过访问 T 中的各种工具（例如电子邮件客户端、文件浏览器和金融应用程序）来增强任务解决能力。每个工具都是一个函数，它接受类型化的参数并返回一个字符串形式的观察结果，如图 2a 所示。工具可以以不同的粒度级别进行实例化，从调用整个应用程序到在生成的代码中调用 API。它们的执行方式决定了代理如何与外部环境交互。

AI 代理的开发很复杂，涉及各种模块、战略架构决策和复杂的实现 [46]。我们的公式将代理视为黑箱，从而可以适应各种设计选择，无论是利用单个 LLM [40]、多个 LLM [53] 还是记忆组件 [42]。

唯一的要求是代理可以调用 T 中的工具。

3.2 威胁模型

攻击者的目标。攻击者的目标是扰乱智能体的任务求解流程，导致智能体执行未经授权的操作，从而使攻击者获利。由于智能体通过工具调用与外部环境交互，因此这种危险行为在算法 1 的第 3 行表现为恶意工具调用。鉴于工具调用可能产生的结果范围很广，攻击者可能会造成各种下游损害。例如，如 [11, 57] 所示，攻击者可以诱发危险的数据擦除操作和未经授权的金融交易。

攻击者能力。我们假设攻击者可以操纵环境 E 中代理的外部数据源，例如邮件，以嵌入恶意命令。代理通过工具调用检索这些数据时，注入命令可能改变其行为。用户 U 则被假设为良性：初始查询 o_0 始终良性，而观测 o_i（i > 0）可能由攻击者控制。但是，攻击者不能修改代理内部结构，例如模型或系统提示，因为现实中的代理通常对外部人员是黑盒。

Progent 的防御范围。Progent 旨在为 AI 代理的工具调用强制执行权限控制策略提供一个通用框架。正如我们在评估部分（第 8 节）所示，它有助于在各种场景下有效地保护代理。然而，它存在局限性，无法处理某些类型的攻击，这些攻击明确超出了本文的研究范围，可以作为未来值得探讨的研究方向。首先，Progent 无法防御那些在完成用户任务所需的最小权限范围内运行的攻击。例如，偏好操纵攻击，攻击者诱骗代理在有效选项中优先选择攻击者的产品 [34]。其次，由于 Progent 侧重于限制工具调用，因此它无法处理针对文本输出的攻击。

4. Progent 的特权控制策略

本节介绍 Progent 的权限控制策略。我们给出其定义（第 4.1 节），描述其在运行时如何强制执行（第 4.2 节），并展示如何通过 SMT 求解比较两种策略，以确定一种策略是另一种策略的扩展还是缩减（第 4.3 节）。

4.1 策略定义

如图 2b 所示，Progent 的权限控制策略提供了一种表达力强且功能强大的权限控制方式。Progent 策略 P 是一个规则列表，它共同描述了代理可以发出的工具调用集。每条规则 $R \in P$ 都针对一个特定的工具，并根据其参数指定允许或禁止工具调用的条件。规则的“回退”操作可以描述如何处理被阻止的工具调用。下文将详细介绍每条规则的核心结构。

效果和条件。图 2b 的‘规则’行以 E t 开头：E 表示允许或禁止，t 是目标工具的标识符。条件序列 e_i 根据工具参数规定何时允许或阻止调用；其重要性在于安全通常依赖具体参数。每个 e_i 是关于第 i 个参数 p_i 的布尔表达式，支持逻辑运算、比较、成员访问（如 $p_i[n]$）、数组长度（如 p_i.length）、成员查询（in）和正则表达式匹配（match）。

Tool definition	$T ::= t\ (\ \overline{p_i : s_i}\ ) :$ `string`
Tool call	$c ::= t\ (\ \overline{v_i}\ ) \mid \{\ t\ (\ \overline{v_i}\ )_j\ \}$
Identifier	t, p
Value type	$s ::=$ `number` | `string` | `boolean` | `array`
Value	$v ::=$ literal of any type in s

(a) A formal definition of tool calls in AI agents.

Policy	$P ::= \overline{R}$
Rule	$R ::= E\ t$ `when` $\{\ \overline{e_i}\ \}$, `fallback` f
Effect	$E ::=$ `allow` | `forbid`
Expression	$e_i ::= v \mid p_i \mid p_i[n] \mid p_i$.`length` | e_i `and` $e'_i \mid e_i$ `or` $e'_i \mid$ `not` $e_i \mid e_i\ bop\ e'_i$
Operator	$bop ::= < \mid \leq \mid == \mid$ `in` | `match`
Fallback	$f ::=$ terminate execution | request user inspection | return msg

Tool identifier t, constant value v,
i-th tool argument p_i, string msg.

(b) A definition of Progent's privilege control policy.

图 2：AI 代理中的工具调用定义和 Progent 的权限控制策略。

回退操作。每条规则都包含一个回退函数 f，当工具调用被禁止时执行该函数。f 的主要目的是引导执行其他操作；它可以向智能体提供关于如何继续的反馈，或者引导用户做出最终决定。我们目前支持三种类型的回退函数，未来可能会添加更多类型：（i）立即终止智能体执行；（ii）通知用户，由用户决定下一步操作；（iii）不执行工具调用，而是返回错误消息 msg。本文默认使用选项 (iii)，并向智能体提供反馈消息“由于 {reason}，工具调用被禁止。请尝试其他工具或参数，并继续完成用户任务：{o}。”。字段 {reason} 因规则而异，用于解释工具调用被禁止的原因，例如，指出其提供的参数如何违反了规则的条件。这起到了一种自动反馈机制的作用，使代理能够调整其策略并继续执行用户的原始任务。例如，在运行示例（图 1）中，当 send_email 在步骤 4 被阻塞时，备用消息允许代理继续执行并成功完成步骤 5 中的正常任务。

4.2 策略运行时

Progent 的运行时强制执行是一个确定性过程，它会根据当前策略检查每个工具调用，如算法 2 所示。

给定策略 P 和工具调用 $c = t(v_i)$，先取 P 中针对工具 t 的规则子集 P_t（第 2 行）。保守地将 forbid 规则排在 allow 规则前，使禁止规则优先生效（第 3 行）。随后遍历排序后的规则 R（第 4 行）。在第 5 行，$e_i[v_i/p_i]$ 表示用运行时的具体参数值 v_i 替换条件中的 p_i，得到布尔结果，判断条件是否满足、规则是否生效。

Algorithm 2: Applying Progent's policy on a tool call.

1 **Procedure** $P(c)$
  Input : Policy P, Tool call $c := t(\overline{v_i})$.
  Output: A secure version of the tool call based on P.
2 P_t = a subset of rules in P that target t
3 Sort P_t such that `forbid` rules comes before `allow` rules
4 **for** R **in** P_t **do**
5 **if** $\overline{e_i}[\overline{v_i}/\overline{p_i}]$ **then**
6 $c' = f$ **if** $E ==$ `forbid` **else** c
7 **return** c'
8 **return** f_{default}

若规则生效，则应用 R：按效果 E 调整调用 c（第 6 行）。若 E 为 forbid，阻止 c，并用回退函数 f 代替；若为 allow，则原样允许 c。在第 7 行返回调整后的 c′ 。若没有针对该工具的规则，或参数未触发任何规则，则默认阻止调用并返回默认回退函数 f_default（第 8 行）。函数 P(c) 得到受策略控制的调用：获准时与原调用 c 相同，否则切换到回退函数。

4.3 通过 SMT 求解进行策略比较

如第 2 节所述，Progent 需要比较两种策略，以确定一种策略是另一种策略的扩展还是缩减。现在，我们将这种比较形式化，并将其简化为 SMT 求解。令 $A(P)$ 表示策略 P 允许的所有工具调用集合：

$$A(P) = \{\, c \mid P(c) = c \,\}, \tag{1}$$

其中 c 是工具调用，P(c) 是算法 2 的输出；P(c) = c 表示允许调用 c，即原样返回。给定 P 和 P′ ，若 A(P) ⊇ A(P′)，则 P′ 是 P 的收窄，包括 A(P) = A(P′) 的情况。反之，若 A(P) ⊉ A(P′)，则 P′ 是 P 的扩展，即它允许 P 不允许的某些调用。图 1c 中，P_3 是 P_2 的收窄（收件人绑定到 alice），P_2 是 P_1 的扩展（增加工具）。

接下来将判定收窄关系 A(P) ⊇ A(P′) 归约为 SMT 求解，等价于检查：∀c. P′ (c) = c ⇒ P(c) = c。每个调用由工具名 t 和参数 v_i 组成，因此等价于：∀t, v_i. P′ (t(v_i)) = t(v_i) ⇒ P(t(v_i)) = t(v_i)。随后逐个工具 t 检查：

$$\forall \overline{v_i}.\ P'_t(t(\overline{v_i})) = t(\overline{v_i}) \Longrightarrow P_t(t(\overline{v_i})) = t(\overline{v_i}). \tag{2}$$

基于算法 2 的循环（第 4 至 7 行），我们有

$$\begin{aligned} P_t(t(\overline{v_i})) = t(\overline{v_i}) \Leftrightarrow\ & (\forall R \in P_t.\ E = \texttt{forbid} \Longrightarrow \neg \overline{e_i}[\overline{v_i}/\overline{p_i}]) \wedge \\ & (\exists R' \in P_t.\ E' = \texttt{allow} \wedge \overline{e'_i}[\overline{v_i}/\overline{p'_i}]). \end{aligned} \tag{3}$$

Algorithm 3: Enforcing Progent's policy at agent runtime.

  Input : User query o_0, agent $\mathcal{A}$, tools $\mathcal{T}$, environment $\mathcal{E}$.
  Output: Agent execution result.
1 initialize P with o_0
2 **for** $i = 1$ ***to*** `max_steps` **do**
3 $c_i = \mathcal{A}(o_{i-1})$
4 **if** c_i is a tool call **then**
5 $o_i = \mathcal{E}(P(c_i))$
6 P' = perform update on P based on o_i
7 **if** $A(P') \subseteq A(P)$ **then**
8 $P = P'$
9 **else** ask for approval
10 **else** task solved, **return** task output
11 task solving fails, **return** unsuccessful

* Green highlights additional modules introduced by Progent.

将式 (3) 的右侧记作 SMT 公式 Φ[P_t](v_i)，则判定式 (2) 等价于求解：

$$\forall \overline{v_i}.\ \Phi_{P'_t}(\overline{v_i}) \Longrightarrow \Phi_{P_t}(\overline{v_i}) \tag{4}$$

5. 使用 Progent 确保代理执行

在第 4 节策略执行的基础上，我们现在讨论 Progent 如何确保代理的完全执行，以及如何使用 LLM 自动生成和更新策略。

使用 Progent 的代理执行循环。算法 3 展示了 Progent 如何集成到标准代理执行循环（算法 1）中，其中附加模块以绿色突出显示。

在第 1 行，根据用户的任务初始化策略 P。

在我们的威胁模型中，如果用户查询是良性的，则初始化的 P 应能根据最小权限原则准确识别并限制工具的使用及其参数。在第 5 行，我们没有执行未受保护的工具调用 c，而是调用了其受策略控制的版本 P(c)。

代理在执行中动态获取新上下文，因此需要更新策略，整合外部信息以满足当前的最小权限要求。但与初始化所用的可信查询 o_0 不同，运行时环境信息未必可信。为避免引入攻击面，算法 3 第 6、7 行将更新分为两步：先生成候选策略 P′ （第 6 行），再用第 4.3 节的 SMT 求解器检查 A(P′) ⊆ A(P)（第 7 行）。若成立，更新是收窄，自动接受；否则是扩展，须获批准。未经批准时，更新只能收窄权限，任何扩展均需显式批准。第 7 节的审批模块可配置扩展处理方式，包括自动拒绝或自动批准。

也支持按工具设置审批规则或人工审核。在图 1c 的示例中，$P_2 \rightarrow P_3$ 是收窄，自动应用；$P_1 \rightarrow P_2$ 是扩展，需要审批。

如第 2.1 节所述，代理的安全性取决于任务和执行上下文，因此为每个可能的用户任务手动制定每一步的策略是不切实际的。为了解决这个问题，我们使用大语言模型 (LLM) 通过推理这些上下文来自动生成和更新策略。具体来说，我们在算法 3 的第 1 行和第 6 行中将 LLM 集成到策略初始化和更新中。我们发现 LLM 能够有效地管理 Progent 的策略，这可能是由于其强大的安全能力。接下来，我们将描述如何在第 1 行和第 6 行中获取初始策略和更新后的策略 P。

初始策略生成。LLM 根据可用工具集 T 和可信的初始用户查询 o_0，理解任务需求并生成特定任务策略 P。在威胁模型中，用户查询良性，因此此阶段只接触可信内容。第 8.2 节实验显示，即使不更新，初始策略也能有效防御多种间接提示注入：攻击成功率从 39.9% 降至 2.5%，同时保持效用。这与模型规划工具调用和生成最小权限策略的能力一致。

策略更新。初始策略已提供强大的防御能力，但动态代理规划引入了策略更新的需求，需要整合外部信息以维持策略的有效性和安全性。与策略初始化不同，第 6 行的策略更新需要整合来自环境的内容，而这些内容不受信任，可能被攻击者篡改。

为防范对抗性输入，候选策略生成分两步。首先，LLM 判断是否可能需要更新，其输入包括工具集 T、初始查询 o_0 和当前工具调用 c_i。此阶段不提供工具返回 o_i，避免判断过程接触潜在恶意内容。如果调用不提供有用信息或与任务无关，例如读取无关文件、写文件或发邮件，则无需更新。如果需要更新，LLM 再根据完整上下文（T、o_0、c_i、返回 o_i 和当前策略 P）生成候选 P'。随后按照前述代理执行流程，用第 4.3 节的 SMT 比较判定其是收窄还是扩展，并分别处理。即使候选 P' 被环境中的对抗性内容影响，这种比较也会检测扩展，阻止它在未经批准时生效。

6. Progent 的安全保障

现在我们阐述算法 3 提供的安全保证。基于 SMT 的扩展检查（第 7 行）确保每次自动应用的策略更新都满足以下条件：

$$A(P') \subseteq A(P), \tag{5}$$

其中 $A(P)$ 是第 4.3 节中定义的代理允许的工具调用。

由于公式 (5) 在每个更新步骤中对每个工具都强制执行（算法 3 中的第 7 至 9 行），因此允许的工具调用集在已批准的扩展之间形成单调递减序列：

$$A(P^{(0)}) \supseteq A(P^{(1)}) \supseteq A(P^{(2)}) \supseteq \dots \tag{6}$$

这一性质称为单调约束：未经显式批准，代理的动作空间不能扩大。□ 图 1 中，$P_1 \rightarrow P_2$ 是需要审批的扩展。批准后，不变量保证 $A(P_2) \supseteq A(P_3) = A(P_4) = A(P_5)$，即两次审批之间权限只能减少。

7.实施

Progent 的特权控制策略。我们实施 Progent 的

我们使用 JSON Schema [19] 来制定策略。最先进的 LLM 服务提供商（例如 OpenAI API [35]）使用 JSON 来格式化工具调用，因此 JSON Schema 自然成为验证工具调用是否符合策略规则的理想选择。由于 JSON 在 LLM 训练数据中广泛存在，LLM 无需任何微调即可生成语法正确的 JSON 格式策略，从而实现第 5 节所述的自动化策略生成和更新流程。我们利用 jsonschema 库 [38] 来实现策略执行。我们使用 Z3 SMT 求解器 [9] 来实现第 4.3 节所述的策略比较。

模块化集成。得益于模块化设计，Progent 可以通过在代理及其工具之间插入策略检查来实现集成，无需修改代理的内部架构。我们提供两种互补的集成模式：一种是库模式，开发者只需对代理运行时进行少量代码更改即可直接嵌入 Progent；另一种是代理模式，最终用户只需将端点重定向到 Progent 的代理服务器即可透明地保护代理，而无需更改任何代理代码。第 8.3 节提供了与实际代理框架的具体集成示例。

通用策略。在许多部署场景中，不同的利益相关者都有必须适用于所有用户任务的持久安全要求：组织可能强制执行数据治理规则，代理开发人员可能限制对敏感工具的访问，而最终用户可能希望保护其个人数据。Progent 通过通用策略支持这一点，这些策略可以手动指定，并在整个执行过程中保持不变。通用策略 P_g 和任务特定策略通过使用算法 4 按顺序应用来组合：首先评估通用策略；如果它没有明确允许或禁止调用，则应用任务特定策略 P；否则，遵循通用策略的决定。这确保了任何被通用策略禁止的工具调用都会被彻底阻止，并且无法被 P 恢复，从而保证了通用限制：有效动作空间满足 $A(P_g, P) \subseteq A(P_g)$。

当不同利益相关者提供具有不同权限级别的通用策略时，Progent 支持显式优先级的多层策略。按优先级排列的 P_A、P_B、P_C、P_D，以 (((P_A, P_B), P_C), P_D)(c) 的形式应用，其中 P_A 优先级最高；高优先级策略先执行且不可被覆盖。

其中 P 的优先级高于优先级较低的策略，而优先级较低的策略可能会进一步限制剩余的行动空间，从而确保任何优先级较低或特定任务的策略都不会削弱更强的通用保证。

审批模块。默认情况下，当基于 SMT 的扩展检查（第 4.3 节）检测到拟议的更新会扩展策略时，需要确认后才能生效。Progent 包含一个可配置的审批模块，用于控制扩展请求的处理方式。在最粗略的层面上，用户可以根据所需的安全性与实用性之间的权衡，自动批准所有扩展或自动拒绝所有扩展。在更精细的层面上，用户可以定义针对特定工具的审批规则，自动批准特定可信上下文（例如，仅访问可信本地数据的工具）的扩展，同时仍然要求对其他工具进行确认。审批模块还可以配置为要求对初始策略和每个扩展请求进行人工审核，从而为安全关键型部署提供额外的保障。我们将在第 8.2 节中评估这些配置。

Algorithm 4: Applying two policies on a tool call.

Procedure $(P_A, P_B)(c)$
  Input : Policies P_A, P_B (P_A has higher priority than P_B), Tool call $c := t\ (\overline{v_i}\ )$.
  Output : A secure version of the tool call based on P_A and P_B.
  $c'_i = P_A(c_i)$
  if $c'_i == f_{\text{default}}$ **then**
    $c'_i = P_B(c_i)$
  return c'_i

在此模式下，有意避免手动禁止任何工具或工具调用参数，而是由自动化流程生成并更新相应的策略。此设置代表了一种最坏情况：用户既未定义可信参数列表，也未在通用策略中限制敏感工具，并且始终选择批准策略扩展请求，从而模拟安全意识有限、在实践中可能做出冒险决策的用户。

我们将 Progent 与 AgentDojo [11] 原始论文中实现的四种先前的防御机制以及两种最先进的防御机制进行了比较：(i) repeat_user_prompt [24] 在每次工具调用后重复用户查询；(ii) spotlighting_with_delimiting [17] 使用特殊分隔符格式化所有工具调用结果，并提示代理忽略这些分隔符内的指令；(iii) tool_filter [43] 在代理执行之前提示 LLM 提供解决用户任务所需的一组工具，并从代理可用的工具集中移除其他工具；(iv) transformers_pi_detector [37] 使用在 DeBERTa [16] 上微调的分类器来检测每次工具调用结果中的提示注入，如果检测到注入则中止代理；(v) DataSentinel [31] 是一个基于博弈论微调的检测器；（vi）LlamaPromptGuard2 [32] 是 Llama 团队提供的提示注入检测器。

图 3 展示了 Progent、先前的防御措施以及未应用任何防御措施的基线在 AgentDojo 上的测试结果。Progent 将 ASR 率从无防御基线的 39.9% 显著降低至 1.0%，同时在无攻击和受攻击场景下均保持了效用评分。这意味着 Progent 成功地执行了最小权限原则，允许完成用户任务所需的工具调用，同时阻止恶意工具调用。该对比也突显了 Progent 相对于先前防御机制的整体优势。tool_filter 的效用降低和 ASR 率更高，因为它采用的忽略工具参数的粗粒度方法要么会完全阻止工具运行，从而损害效用，要么会完全允许工具运行，导致攻击成功。我们还观察到，三种提示注入检测器（transformers_pi_detector、DataSentinel 和 LlamaPromptGuard2）效果不佳。虽然它们在与其训练分布相似的数据集上可能表现良好，但它们无法泛化到 AgentDojo 数据集，表现出较高的误报率和漏报率。

ASB。与 AgentDojo 类似，我们采用基于 LLM 的自主策略生成方法。也就是说，我们利用了第 5 节中描述的方法，但不提供任何手动定义的通用策略来模拟最坏情况。我们将 Progent 与原始论文中实现的先前防御措施进行了比较。

8 实验评估

本节对 Progent 进行了全面的评估。我们首先在两个常用的基准测试中展示了 Progent 的总体有效性（第 8.1 节），然后分析了其各个组件的影响（第 8.2 节），最后在真实世界的智能体和多智能体系统上评估了 Progent（第 8.3 节）。

8.1 Progent 的总体有效性

评估设置。我们使用两个常用的基准测试来评估 Progent。第一个是 AgentDojo [11]，这是一个先进的提示注入基准测试。AgentDojo 包含日常生活中常见的四种代理使用场景：（i）银行业务：执行银行相关操作；（ii）Slack：处理 Slack 消息、读取网页和文件；（iii）旅行：查找和预订航班、餐厅和租车；（iv）工作区：管理电子邮件、日历和云盘。攻击者在环境中注入恶意提示，这些提示通过工具调用返回到代理的工作流中，从而指示代理执行攻击任务。第二个基准测试是 ASB [57]，它也考虑了通过环境的间接提示注入，类似于 AgentDojo。ASB 提供了十个代理场景和五个攻击模板。我们评估防御的两个关键方面：效用，以代理完成良性用户任务的成功率来衡量（无论是否受到攻击都会报告），以及安全性，以攻击成功率 (ASR) 来衡量。

我们始终使用 gpt-4o 作为代理的底层大语言模型 (LLM) 以及策略生成和更新的 LLM。对于扩展审批，我们假设用户将审批者配置为始终接受策略扩展请求，以此模拟用户安全意识有限且在实践中可能做出冒险决策的最坏情况。我们在第 8.2 节中探讨了不同的审批者配置和不同的模型选择。

AgentDojo。为了在 AgentDojo [11] 的四个不同的代理类别中实例化 Progent，我们将通用权限控制策略与 LLM 生成并动态更新的特定任务策略相结合，如第 5 节和第 7 节所述。对于通用策略，我们全局允许使用代理通常用于收集任务规划信息的非敏感只读工具。

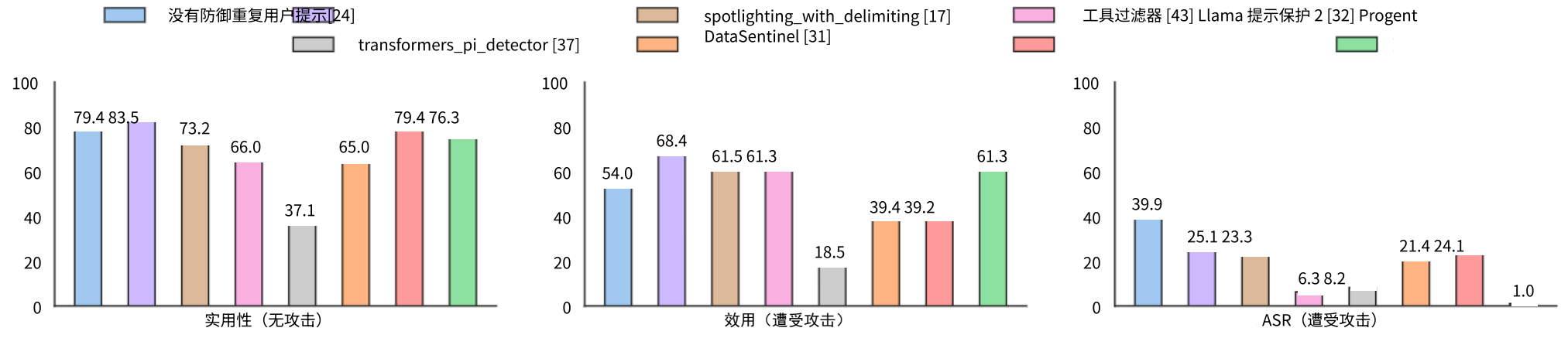


图 3：AgentDojo 上的基线代理（无防御）、先前的防御和 Progent 之间的比较 [11]。

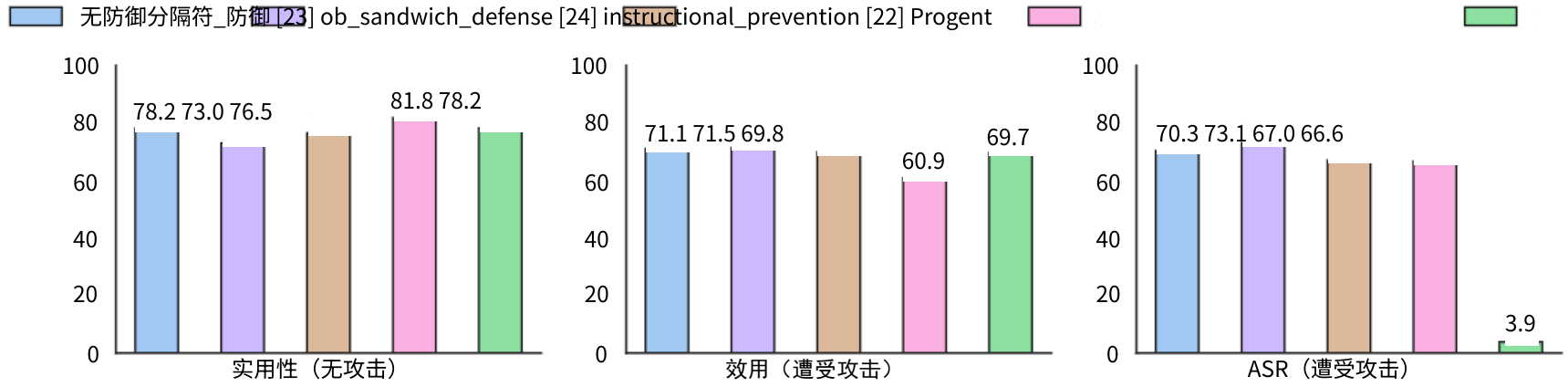


图 4：ASB 的比较结果 [57]。

ASB [57]：（i）delimiters_defense [23] 使用分隔符包装用户查询，并提示代理仅执行分隔符内的用户查询；（ii）ob_sandwich_defense [24] 在工具调用结果的末尾附加一个包含用户任务的附加指令提示；（iii）instructional_prevention [22] 重构用户查询，并要求代理忽略除用户任务之外的所有命令。

图 4 展示了 ASB 上的对比结果。Progent 保持了与无防御设置相当的效用得分。这是因为我们的策略不会阻止代理完成良性用户任务所需的正常功能。具体来说，LLM 生成的策略能够成功识别用户任务所需的工具并允许其使用。Progent 还将 ASR 从 70.3% 显著降低至 3.9%。我们进一步研究了 LLM 生成策略的失败案例。这些失败案例大多是由于攻击工具调用的名称和描述与良性工具的名称和描述非常相似，并且与用户任务密切相关。因此，我们认为即使对于人类而言，在没有预先了解哪些工具调用可信的情况下，也很难识别这些攻击工具调用。我们还尝试使用 Progent 的策略来表达经过人工补充的规则。这些手动定义的策略能够有效地将 ASR 降低至 0%。之前的防御措施无法有效降低 ASR，这一结果与 ASB 的原始论文 [57] 一致。

8.2 消融研究

不同的审批人配置。如第 7 节所述，Progent 的审批人模块控制着策略扩展的处理方式。

为了系统地探讨自动化与安全性之间的权衡，我们比较了四种具有代表性的审批配置。前三种配置完全自动化，而第四种配置则需要人工审核：（i）禁用更新：LLM 生成初始策略，并自动接受；策略更新机制被禁用，因此初始策略在整个执行过程中保持不变；（ii）自动拒绝：LLM 基于用户提出的良性查询生成初始策略，并自动接受；在执行过程中，所有缩小范围的更新都会被应用，但所有扩展更新都会被拒绝；（iii）自动批准：LLM 基于用户提出的良性查询生成初始策略，并自动接受；在执行过程中，所有更新（包括缩小范围和扩展更新）都会被自动接受；（iv）人工审批：人工审核初始策略以及执行过程中的每一次扩展更新，并从策略中删除任何不合适的规则。人工审批流程的详细信息见附录 D。

在这些配置中，“自动批准”代表了最坏的情况，即所有扩展都未经审核即被接受。为了评估这种最坏情况，我们在其他实验中采用了“自动批准”模式作为默认配置。相比之下，“自动拒绝”模式在安全性和自动化之间提供了一种较为保守的平衡，确保所有扩展都被拒绝。“禁用更新”模式通过在初始化后固定策略来提供确定性的稳定性，但可能会降低对动态环境的适应性。最后，“手动批准”模式提供了最强的安全性，因为所有策略都由用户显式审核，但这需要人工干预，最适合安全至关重要的环境。

图 5 展示了 Progent 在不同配置下的性能。在禁用更新模式下，与无防御基线　相比，ASR 得到了有效降低。这是因为

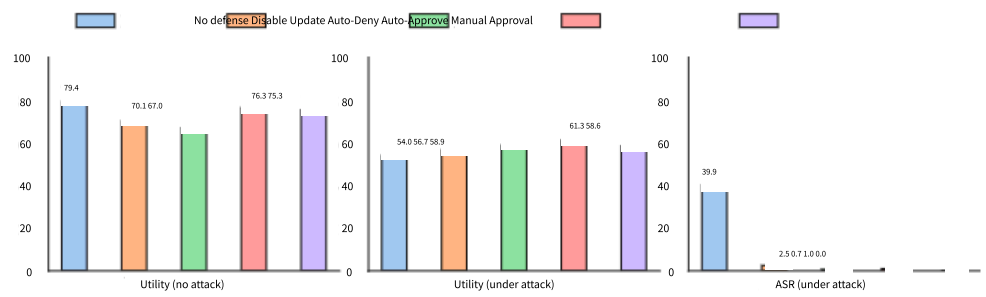


图 5：Progent 在不同配置下的有效性。

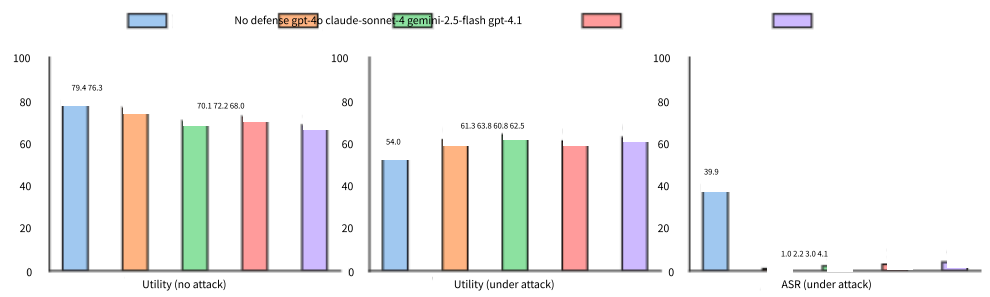


图 6：Progent 对不同策略 LLM 的有效性。

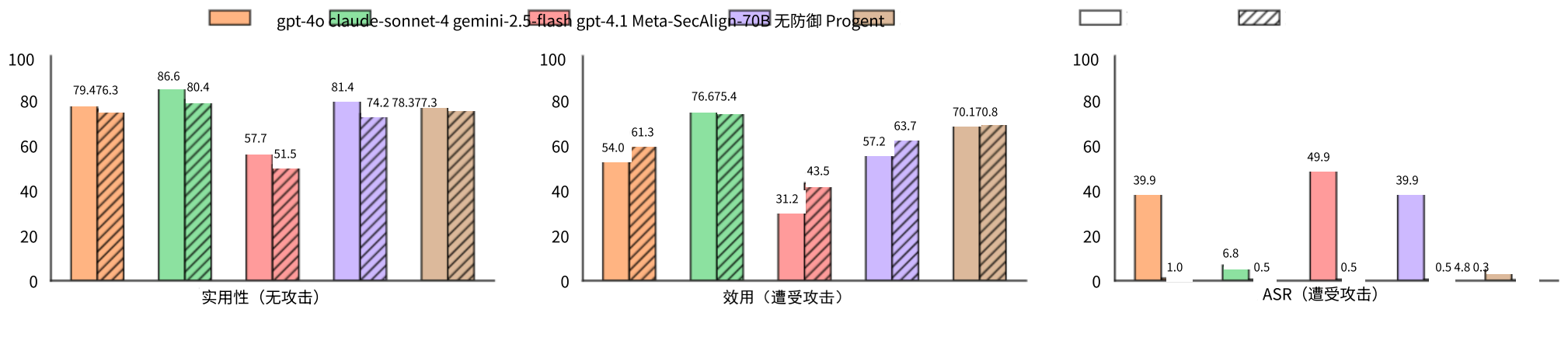


图 7：Progent 对不同代理 LLM 的有效性。

由于许多任务在用户查询中都具有明确的目标，因此攻击成功率 (ASR) 显著降低，仅基于初始上下文生成的策略即可提供强大的安全保护。启用策略更新机制后，在所有全自动配置中，“自动拒绝”模式实现了最高的安全性，因为所有更新都严格受到不变性检查的限制。相比之下，“自动批准”模式允许策略扩展到其初始不变性之外，与“禁用更新”模式相比，这提高了实用性和安全性。然而，“自动批准”模式的 ASR 略高于“自动拒绝”模式，因为扩展的策略可能会无意中允许恶意行为，因此在实践中需要对此类更新进行显式批准。最后，“手动批准”模式通过人工审核将 ASR 降低至 0%，这表明人工监督可以捕捉到自动流程可能遗漏的复杂安全需求。这一结果突显了防御自动化和安全保障之间的权衡，并展示了 Progent 在支持自动和手动审批配置方面的优势。

我们还观察到，在基于 SMT 的扩展检查之后，只有 6%的策略更新是需要审批的扩展；其余的都是自动处理的缩减。这使得 Progent 能够在保持高安全性的同时，将审批开销降至最低。

不同的模型选择。在本实验中，我们首先探索了第 5 节讨论的自动化策略生成和更新方法的模型选择。我们在 AgentDojo 中运行代理，使用不同的策略 LLM，同时将代理的底层 LLM 固定为 gpt-4o。如图 6 所示，Progent 在使用不同的策略 LLM 进行策略生成和更新时均有效，在所有模型中将 ASR 降低到 5% 以下，使用性能最佳的 LLM 时甚至降低到 1%。

接下来，我们将 gpt-4o 设置为策略 LLM，并运行具有不同底层代理 LLM 的代理。我们将无防御基线与使用 gpt-4o 生成和更新策略进行比较。如图 7 所示，Progent 在不同的代理 LLM 上均有效。

在未受攻击的情况下，Progent 要么保持效用不变，要么仅略微降低效用。在遭受攻击的情况下，它能提升效用，并显著降低不同模型的攻击成功率 (ASR)。我们还发现，claude-sonnet-4 和 Meta-SecAlign-70B 本身就具有强大的安全机制，在未应用任何防御措施的情况下，它们的攻击成功率分别仅为 6.8% 和 4.8%，表现出色。应用 Progent 后，攻击成功率进一步降低至 0.5% 和 0.3%，能够抵御约 90% 的攻击。这些结果表明，Progent 可以作为现有防御机制的补充，并在实践中发挥有效的纵深防御作用。

8.3 真实世界代理集成

为了评估真实世界的智能体和多智能体系统，我们首先构建了一个基于 MCP 的基准测试。然后，我们将 Progent 集成到真实世界的智能体和多智能体系统中，以展示其在实际部署场景中的性能和实用性。我们使用多种真实世界的智能体开发框架和智能体产品对 Progent 进行了评估，包括 LangChain [21]、OpenAI Agents SDK [36]、OpenHands [48]和 AutoGen [53]。集成细节见附录 F。

基于 MCP 的基准测试。第 8 节中使用的基准测试均采用各自的代理实现。为了进一步展示 Progent 在更复杂的真实场景中的实用性和有效性，我们将其集成到流行的代理开发框架和代理产品中。

为了评估这些真实世界的智能体，我们需要一个沙盒环境，用于安全交互和逼真的攻击模拟。为此，我们将 AgentDojo 的环境和工具，以及所有良性任务和攻击任务，迁移到 MCP 服务器，构建了一个能够直接与真实世界智能体交互的全新基准测试平台。这套基于 MCP 的统一工具套件提供了一个标准化的环境，用于模拟真实场景并评估真实世界智能体的性能。它完全可重用，并且可以应用于未来的智能体框架或产品。

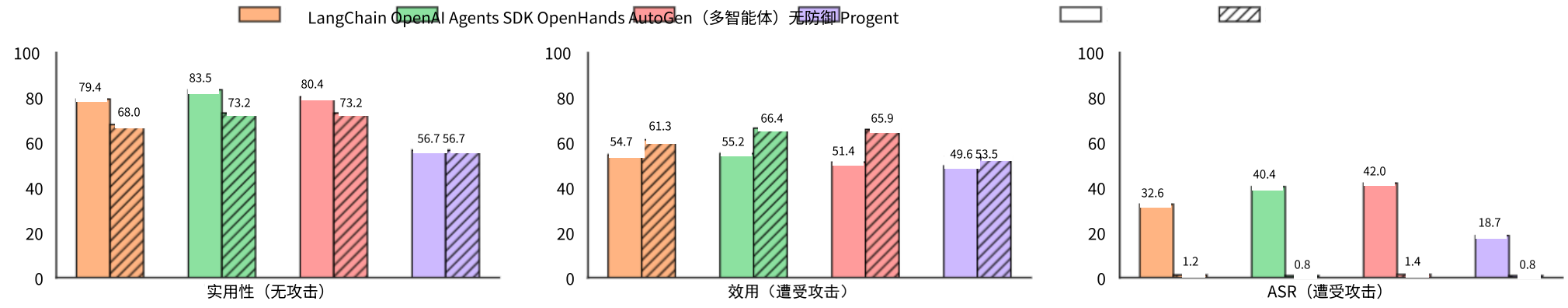


图 8：Progent 在现实世界智能体和多智能体系统中的有效性。

因为它们支持 MCP，使其成为现实世界代理安全研究的实用且可扩展的测试平台。

真实世界智能体。在本实验中，我们将 Progent 应用于两个流行的智能体开发框架（LangChain 和 OpenAI Agents SDK）以及一个智能体产品（OpenHands）。如第 7 节所述，Progent 提供了两种集成模式，分别针对不同的用户角色：面向智能体开发者的库模式和面向最终用户的代理模式。我们对 LangChain 和 OpenAI Agents SDK 采用库模式，模拟开发者将 Progent 集成到现有智能体中，并仅需对现有智能体进行少量代码修改。具体来说，对于 LangChain，我们实现了一个中间件组件，开发者在使用 LangChain SDK 定义智能体时可以轻松添加该组件。对于 OpenAI Agents SDK，我们提供了一个 Agent 类的包装器，使开发者能够将 Progent 的功能无缝集成到他们的智能体中。对于 OpenHands，我们采用代理模式，模拟最终用户透明地应用 Progent，而无需更改智能体的内部实现。在这种模式下，最终用户无需修改智能体的代码；相反，他们只是将 LLM 和 MCP 服务器端点更改为 Progent 的代理，以在现有代理产品上启用其保护功能。

如图 8 所示，Progent 能够将 ASR 稳定降低至 1% 左右，同时保持较高的效用。不同代理之间效用和安全性的细微差异可能是由于这些代理的系统提示和重试机制存在差异所致。这些结果表明，在实际攻击场景下，Progent 与真实代理配合使用时，展现出卓越的有效性和鲁棒性。

多智能体系统。Progent 的模块化设计使其天然支持多智能体系统。我们提出了两种将 Progent 集成到多智能体环境中的方法。第一种方法将整个多智能体系统视为一个统一的实体，并在多智能体系统、环境和 LLM 之间的接口上强制执行单一的权限控制层。这种全局上下文和策略管理确保了安全性的一致性，并防止跨智能体权限提升。Progent 也可以单独应用于每个子智能体，从而允许根据每个智能体的角色和功能定制策略。当不同的智能体由不同的用户操作或拥有显著不同的权限级别，需要隔离的权限控制时，这种设置尤其有用。

对于我们基于 MCP 的基准测试，我们构建了一个包含 AutoGen 的多智能体系统，该系统由一个协调器和六个专家组成。子代理包括：(i) 管理用户个人资料和密码的通用代理；(ii) 负责日历、电子邮件和文件操作的工作区代理；(iii) 处理团队沟通和频道管理的 Slack 代理；(iv) 执行银行和支付相关任务的银行代理；(v) 管理预订和行程的旅行代理；以及 (vi) 处理网页浏览和内容发布的 Web 代理。在本实验中，我们采用第一种集成方法将 Progent 集成到多代理系统中，因为所有子代理都代表同一用户运行，所以统一控制更为合适。

图 8 显示，在多智能体系统中，效用和攻击成功率 (ASR) 均略低于单智能体系统。这可能是因为多智能体系统自然地引入了子智能体之间的隔离，虽然这会降低协调效率，但也能限制攻击传播，从而提高安全性。应用 Progent 后，系统在保持相近效用的同时，显著提高了安全性，能够抵御超过 95% 的成功攻击。

9. 局限性与讨论

用户错误。尽管 Progent 采用了基于确定性 SMT 的检查机制，但用户错误仍然可能危及它的安全保障。如果用户错误地批准了权限扩展更新或给出了模糊的初始任务，Progent 可能会允许超出最低权限范围的策略。我们的策略更新机制、强制执行不变约束以及要求对潜在的不安全变更进行显式批准，可以缓解但无法消除这些风险。在附录 E 中，我们使用自适应攻击对 Progent 进行了实证评估，以应对用户引入的错误。结果表明，即使存在人为误判，Progent 也能显著降低攻击成功率 (ASR) 并保持有效的保护。

防御范围。Progent 专注于防御超出不变边界的恶意工具调用。因此，Progent 无法应对仅影响代理文本输出而不触发危险工具调用的攻击。此类文本到文本的攻击更接近于模型级风险，而现有模型安全工作已对此进行了广泛研究。对于代理而言，Progent 可作为有效的纵深防御层，与其他防御措施相辅相成，且只需极少的代码更改。如果开发人员选择向其代理注入其他组件，则可以通过增强策略来限制工具读取潜在的恶意数据或过滤工具输出以移除有害内容，从而进一步控制数据流并帮助缓解此类攻击。此外，代理模式提供开箱即用的部署功能，无需任何额外配置。

除了修改底层代理之外，它无法保护作为内置工具实现的、绕过外部 MCP 接口的代理功能。相比之下，库模式只需几行代码更改即可保护内置工具。这体现了部署便捷性和保护深度之间的权衡。

扩展到多模态代理。在我们目前的范围内，代理仍然只能处理文本。因此，我们的方法无法应用于包含图形界面等多模态元素的调用工具的代理。代理操作的示例包括点击浏览器中的特定位置[29, 50, 55]或计算机屏幕上的特定图标[58]。一个有趣的未来工作方向是探索设计能够捕获其他模态（例如图像）的策略。例如，该策略可以限制代理只能点击计算机上的特定应用程序。这可以转化为计算机屏幕上的特定区域，代理只能点击该选定区域。

10 相关工作

安全策略语言。实施安全原则极具挑战性，而以往的研究表明，编程是一种可行的解决方案。

Binder [12] 使用 Datalog 风格的推理来实现分布式系统中的授权和委派，而 Sapper [27] 则引入了硬件级检查，以实现对时间敏感的非干扰性。在云和应用层面，Cedar [8] 提供了一种领域特定语言来表达细粒度的授权策略，而诸如 AWS [1]、Microsoft Azure [33] 和 Google Cloud [13] 等主流云平台也提供了成熟的授权策略语言。这些方法表明，程序化策略执行已在不同的安全领域日趋成熟，但现有的策略语言并不适用于人工智能代理的动态行为。这种差距使得面向代理的策略（例如 Progent）成为自然而然的下一步发展方向。

面向智能体的系统级防御。开发系统级

针对智能体任务解决的防御机制是一个新兴的研究领域。IsolateGPT [54] 提出了一种智能体架构，该架构隔离了跨应用程序的执行，并要求用户确认潜在的危险操作（例如，跨应用程序通信或不可逆操作）。f-secure [51] 则通过手动分配的信任标签来强制执行信息流控制，这些标签会在智能体执行期间传播。AirgapAgent [2] 专注于对私有数据的访问控制，确保智能体仅使用与任务相关的信息，但无法控制智能体在数据访问之外的恶意行为。CaMeL [10] 从受信任的用户查询中提取控制流和数据流，以防止不受信任的数据影响程序流程，但这需要修改智能体的内部架构，并且会影响其效用。FIDES [7] 使用带有机密性和完整性标签的动态污点跟踪来规范智能体操作中的数据流，但这需要预先为数据源分配标签。 Conseca [44] 和 DRIFT [25] 也提出了为代理生成策略的方法，但它们都依赖于 LLM 来生成策略，而没有进行确定性验证。Progent 的策略结构支持基于 SMT 的扩展检查，该检查能够确定性地将每次更新分类为权限的扩展或缩减。在 Progent 中即使 LLM 生成的策略更新被恶意输入操纵，扩展检查也能防止任何静默权限提升，从而实现 Conseca 和 DRIFT 都无法提供的单调约束保证。Progent 引入的权限控制方法补充了这些防御措施。其模块化设计使其能够以最小的改动集成到现有的代理实现中，从而可能降低采用门槛，而许多其他系统级防御措施则需要进行大量的架构修改。

模型级提示注入防御。另一条研究路线侧重于在模型层面解决提示注入问题，这可以分为两类。第一类是训练和部署防护模型来检测注入的内容[18, 26, 31, 32, 37]。如图 3 所示，Progent 在实验中优于目前最先进的防护方法[31, 32, 37]。另一个关键区别在于，Progent 提供了确定性的安全强制执行，而防护模型无法做到这一点。第二类防御涉及微调代理的 LLM（大语言模型），使其更能抵抗提示注入[3-5, 45]。这些防御措施与 Progent 的系统级权限控制处于不同的层面。因此，Progent 可以与模型级防御协同工作：模型级防御保护代理的核心推理，而 Progent 则保护代理与外部工具之间的执行边界。如图 7 所示，结合 Progent 和模型级防御 [5] 可以提供更强大的保护。

11 结论

我们提出 Progent，通过工具调用级别的权限控制保护 AI 代理。它用针对工具名称和参数的符号规则表示策略，并进行确定性强制执行，工具调用判定过程不调用 LLM。为应对不同任务和变化的执行上下文，LLM 自动生成及更新策略；基于 SMT 的确定性检查判断更新是收窄（自动应用）还是扩展（需要批准），保证单调约束。AgentDojo 和 ASB 的评估显示，Progent 在保持高效用的同时显著降低攻击成功率；与 LangChain 和 OpenAI Agents SDK 等框架的集成，也证实其作为非侵入式纵深防御层的实用性。

参考

[1] Amazon Web Services. 2025. AWS Identity and Access Management (IAM). https://aws.amazon.com/iam/. Accessed: 2025-04-12.

[2] Eugene Bagdasarian, Ren Yi, Sahra Ghalebikesabi, Peter Kairouz, Marco Gruteser, Sewoong Oh, Borja Balle, and Daniel Ramage. 2024. Airgapagent: Protecting privacy-conscious conversational agents. In *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*. 3868–3882.

[3] Sizhe Chen, Julien Piet, Chawin Sitawarin, and David Wagner. 2025. StruQ: Defending against prompt injection with structured queries. In *USENIX Security Symposium*.

[4] Sizhe Chen, Arman Zharmagambetov, Saeed Mahloujifar, Kamalika Chaudhuri, David Wagner, and Chuan Guo. 2025. SecAlign: Defending Against Prompt Injection with Preference Optimization. In *The ACM Conference on Computer and Communications Security (CCS)*.

[5] Sizhe Chen, Arman Zharmagambetov, David Wagner, and Chuan Guo. 2025. Meta SecAlign: A Secure Foundation LLM Against Prompt Injection Attacks. *arXiv preprint arXiv:2507.02735* (2025).

[6] Sarthak Choudhary, Divyam Anshumaan, Nils Palumbo, and Somesh Jha. 2025. How Not to Detect Prompt Injections with an LLM. *arXiv preprint arXiv:2507.05630* (2025).

[7] Manuel Costa, Boris Köpf, Aashish Kolluri, Andrew Paverd, Mark Russinovich, Ahmed Salem, Shruti Tople, Lukas Wutschitz, and Santiago Zanella-Béguelin. 2025. Securing ai agents with information-flow control. *arXiv preprint arXiv:2505.23643* (2025).

[8] Joseph W Cutler, Craig Disselkoen, Aaron Eline, Shaobo He, Kyle Headley, Michael Hicks, Kesha Hietala, Eleftherios Ioannidis, John Kastner, Anwar Mamat, et al. 2024. Cedar: A new language for expressive, fast, safe, and analyzable authorization. *Proceedings of the ACM on Programming Languages* 8, OOPSLA1 (2024), 670–697.

[9] Leonardo De Moura and Nikolaj Bjørner. 2008. Z3: An efficient SMT solver. In *TACAS*.

[10] Edoardo Debenedetti, Ilia Shumailov, Tianqi Fan, Jamie Hayes, Nicholas Carlini, Daniel Fabian, Christoph Kern, Chongyang Shi, Andreas Terzis, and Florian Tramèr. 2025. Defeating Prompt Injections by Design. *arXiv preprint arXiv:2503.18813* (2025).

[11] Edoardo Debenedetti, Jie Zhang, Mislav Balunovic, Luca Beurer-Kellner, Marc Fischer, and Florian Tramèr. 2024. AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents. In *The Thirty-eight Conference on Neural Information Processing Systems Datasets and Benchmarks Track*.

[12] John DeTreville. 2002. Binder, a logic-based security language. In *Proceedings 2002 IEEE Symposium on Security and Privacy*. IEEE, 105–113.

[13] Google Cloud. 2025. Identity and Access Management (IAM). https://cloud.google.com/iam/. Accessed: 2025-04-12.

[14] Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. 2023. Not what you've signed up for: Compromising real-world llm-integrated applications with indirect prompt injection. In *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security*. 79–90.

[15] Feng He, Tianqing Zhu, Dayong Ye, Bo Liu, Wanlei Zhou, and Philip S Yu. 2024. The emerged security and privacy of llm agent: A survey with case studies. *arXiv preprint arXiv:2407.19354* (2024).

[16] Pengcheng He, Xiaodong Liu, Jianfeng Gao, and Weizhu Chen. 2021. Deberta: Decoding-enhanced bert with disentangled attention. In *ICLR*.

[17] Keegan Hines, Gary Lopez, Matthew Hall, Federico Zarfati, Yonatan Zunger, and Emre Kiciman. 2024. Defending against indirect prompt injection attacks with spotlighting. *arXiv preprint arXiv:2403.14720* (2024).

[18] Hakan Inan, Kartikeya Upasani, Jianfeng Chi, Rashi Rungta, Krithika Iyer, Yuning Mao, Michael Tontchev, Qing Hu, Brian Fuller, Davide Testuggine, et al. 2023. Llama guard: Llm-based input-output safeguard for human-ai conversations. *arXiv preprint arXiv:2312.06674* (2023).

[19] JSON Schema. 2025. JSON Schema. https://json-schema.org/. Accessed: 2025-01-10.

[20] LangChain. 2025. Gmail Toolkit. https://python.langchain.com/docs/integrations/tools/gmail/. Accessed: 2025-01-10.

[21] LangChain. 2025. LangChain. https://github.com/langchain-ai/langchain. Accessed: 2025-01-10.

[22] Learn Prompting. 2024. Instruction Defense. https://learnprompting.org/docs/prompt_hacking/defensive_measures/instruction. Accessed: 2025-08-24.

[23] Learn Prompting. 2024. Random Sequence Enclosure. https://learnprompting.org/docs/prompt_hacking/defensive_measures/random_sequence. Accessed: 2025-08-24.

[24] Learn Prompting. 2024. Sandwich Defense. https://learnprompting.org/docs/prompt_hacking/defensive_measures/sandwich_defense. Accessed: 2025-08-24.

[25] Hao Li, Xiaogeng Liu, Hung-Chun Chiu, Dianqi Li, Ning Zhang, and Chaowei Xiao. 2025. Drift: Dynamic rule-based defense with injection isolation for securing llm agents. *arXiv preprint arXiv:2506.12104* (2025).

[26] Rongchang Li, Minjie Chen, Chang Hu, Han Chen, Wenpeng Xing, and Meng Han. 2024. GenTel-Safe: A Unified Benchmark and Shielding Framework for Defending Against Prompt Injection Attacks. *arXiv preprint arXiv:2409.19521* (2024).

[27] Xun Li, Vineeth Kashyap, Jason K Oberg, Mohit Tiwari, Vasanth Ram Rajarathinam, Ryan Kastner, Timothy Sherwood, Ben Hardekopf, and Frederic T Chong. 2014. Sapper: A language for hardware-level security policy enforcement. In *Proceedings of the 19th international conference on Architectural support for programming languages and operating systems*. 97–112.

[28] Yuanchun Li, Hao Wen, Weijun Wang, Xiangyu Li, Yizhen Yuan, Guohong Liu, Jiacheng Liu, Wenxing Xu, Xiang Wang, Yi Sun, et al. 2024. Personal llm agents: Insights and survey about the capability, efficiency and security. *arXiv preprint arXiv:2401.05459* (2024).

[29] Zeyi Liao, Lingbo Mo, Chejian Xu, Mintong Kang, Jiawei Zhang, Chaowei Xiao, Yuan Tian, Bo Li, and Huan Sun. 2025. Eia: Environmental injection attack on generalist web agents for privacy leakage. *ICLR* (2025).

[30] Yi Liu, Gelei Deng, Yuekang Li, Kailong Wang, Zihao Wang, Xiaofeng Wang, Tianwei Zhang, Yepang Liu, Haoyu Wang, Yan Zheng, et al. 2023. Prompt Injection attack against LLM-integrated Applications. *arXiv preprint arXiv:2306.05499* (2023).

[31] Yupei Liu, Yuqi Jia, Jinyuan Jia, Dawn Song, and Neil Zhenqiang Gong. 2025. DataSentinel: A Game-Theoretic Detection of Prompt Injection Attacks. *Proceedings 2025 IEEE Symposium on Security and Privacy* (2025).

[32] Meta. 2025. Llama Prompt Guard 2. https://www.llama.com/docs/model-cards-and-prompt-formats/prompt-guard/. Accessed: 2025-08-14.

[33] Microsoft. 2025. Azure Policy Documentation. https://learn.microsoft.com/en-us/azure/governance/policy/. Accessed: 2025-04-12.

[34] Fredrik Nestaas, Edoardo Debenedetti, and Florian Tramèr. 2025. Adversarial search engine optimization for large language models. In *ICLR*.

[35] OpenAI. 2025. Function calling – OpenAI API. https://platform.openai.com/docs/guides/function-calling. Accessed: 2025-01-10.

[36] OpenAI. 2025. OpenAI Agents SDK. https://github.com/openai/openai-agents-python. Accessed: 2025-11-10.

[37] ProtectAI.com. 2024. Fine-Tuned DeBERTa-v3-base for Prompt Injection Detection. https://huggingface.co/ProtectAI/deberta-v3-base-prompt-injection-v2.

[38] python-jsonschema. 2025. python-jsonschema/jsonschema – GitHub. https://github.com/python-jsonschema/jsonschema. Accessed: 2025-01-10.

[39] Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, et al. 2023. Toolllm: Facilitating large language models to master 16000+ real-world apis. *arXiv preprint arXiv:2307.16789* (2023).

[40] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. 2023. Toolformer: Language models can teach themselves to use tools. In *NeurIPS*.

[41] Wenqi Shi, Ran Xu, Yuchen Zhuang, Yue Yu, Jieyu Zhang, Hang Wu, Yuanda Zhu, Joyce Ho, Carl Yang, and May Dongmei Wang. 2024. Ehragent: Code empowers large language models for few-shot complex tabular reasoning on electronic health records. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*. 22315–22339.

[42] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. Reflexion: Language agents with verbal reinforcement learning. In *NeurIPS*.

[43] Simon Willison. 2023. The Dual LLM pattern for building AI assistants that can resist prompt injection. https://simonwillison.net/2023/Apr/25/dual-llm-pattern/. Accessed: 2025-08-24.

[44] Lillian Tsai and Eugene Bagdasarian. 2025. Contextual Agent Security: A Policy for Every Purpose. In *Proceedings of the 2025 Workshop on Hot Topics in Operating Systems*. 8–17.

[45] Eric Wallace, Kai Xiao, Reimar Leike, Lilian Weng, Johannes Heidecke, and Alex Beutel. 2024. The instruction hierarchy: Training llms to prioritize privileged instructions. *arXiv preprint arXiv:2404.13208* (2024).

[46] Lei Wang, Chen Ma, Xueyang Feng, Zeyu Zhang, Hao Yang, Jingsen Zhang, Zhiyuan Chen, Jiakai Tang, Xu Chen, Yankai Lin, et al. 2024. A survey on large language model based autonomous agents. *Frontiers of Computer Science* 18 (2024).

[47] Xingyao Wang, Yangyi Chen, Lifan Yuan, Yizhe Zhang, Yunzhu Li, Hao Peng, and Heng Ji. 2024. Executable code actions elicit better llm agents. In *ICML*.

[48] Xingyao Wang, Boxuan Li, Yufan Song, Frank F. Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, Hoang H. Tran, Fuqiang Li, Ren Ma, Mingzhang Zheng, Bill Qian, Yanjun Shao, Niklas Muennighoff, Yizhe Zhang, Binyuan Hui, Junyang Lin, Robert Brennan, Hao Peng, Heng Ji, and Graham Neubig. 2025. OpenHands: An Open Platform for AI Software Developers as Generalist Agents. In *ICLR*. https://openreview.net/forum?id=OJd3ayDDoF

[49] Zhun Wang, Vincent Siu, Zhe Ye, Tianneng Shi, Yuzhou Nie, Xuandong Zhao, Chenguang Wang, Wenbo Guo, and Dawn Song. 2025. AgentVigil: Generic Black-Box Red-teaming for Indirect Prompt Injection against LLM Agents. *arXiv preprint arXiv:2505.05849* (2025).

[50] Chen Henry Wu, Rishi Rajesh Shah, Jing Yu Koh, Russ Salakhutdinov, Daniel Fried, and Aditi Raghunathan. 2024. Dissecting Adversarial Robustness of Multimodal LM Agents. In *NeurIPS 2024 Workshop on Open-World Agents*.

[51] Fangzhou Wu, Ethan Cecchetti, and Chaowei Xiao. 2024. System-Level Defense against Indirect Prompt Injection Attacks: An Information Flow Control Perspective. *arXiv preprint arXiv:2409.19091* (2024).

[52] Fangzhou Wu, Ning Zhang, Somesh Jha, Patrick McDaniel, and Chaowei Xiao. 2024. A new era in llm security: Exploring security concerns in real-world llm-based systems. *arXiv preprint arXiv:2402.18649* (2024).

[53] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Shaokun Zhang, Erkang Zhu, Beibin Li, Li Jiang, Xiaoyun Zhang, and Chi Wang. 2024. Autogen: Enabling next-gen llm applications via multi-agent conversation framework. In *COLM*.

[54] Yuhao Wu, Franziska Roesner, Tadayoshi Kohno, Ning Zhang, and Umar Iqbal. 2025. IsolateGPT: An Execution Isolation Architecture for LLM-Based Systems. In *Network and Distributed System Security Symposium (NDSS)*.

[55] Chejian Xu, Mintong Kang, Jiawei Zhang, Zeyi Liao, Lingbo Mo, Mengqi Yuan, Huan Sun, and Bo Li. 2024. Advweb: Controllable black-box attacks on vlm-powered web agents. *arXiv preprint arXiv:2410.17401* (2024).

[56] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. React: Synergizing reasoning and acting in language models. In *ICLR*.

[57] Hanrong Zhang, Jingyuan Huang, Kai Mei, Yifei Yao, Zhenting Wang, Chenlu Zhan, Hongwei Wang, and Yongfeng Zhang. 2025. Agent security bench (ASB): Formalizing and benchmarking attacks and defenses in llm-based agents. In *ICLR*.
[58] Yanzhe Zhang, Tao Yu, and Diyi Yang. 2024. Attacking Vision-Language Computer Agents via Pop-ups. *arXiv preprint arXiv:2411.02391* (2024).

开放科学

评估中使用的数据集和基准测试均由其作者公开提供。没有任何策略或许可限制阻止我们公开这些资源。

以下是工件的链接：https://github.com/sunblaze-ucb/progent 工件包括：(i) Progent 的实现。(ii) 用于重现第 8 节实验的代码。(iii) 工件的 README 文件和使用说明。

B 伦理考量

本研究符合会议网站和门洛报告的伦理准则。我们的工作重点在于提供一种防御机制，而非攻击方法。我们相信，我们的工作不会造成负面影响，并且有助于提高现有代理系统的安全性。具体而言，我们的方法可以帮助开发者和最终用户更好地控制其代理系统的工具权限。通过本文提出的工具权限控制，用户可以更好地保护其系统免受针对代理的高级攻击。

所有实验均在本地沙盒环境中进行，不会向现实世界的应用程序泄露任何攻击提示。

实验中使用的所有数据集都是公开的，不包含任何私人或敏感数据。

总而言之，据我们所知，这项工作符合伦理道德，我们愿意就任何与伦理问题相关的进一步澄清作出解释。

C 示例策略

我们在图 9 中给出了生成的策略示例。

```
{
  "send_email": [
    {
      "priority": 100,
      "effect": "allow",
      "conditions": {
        "recipients": {
          "type": "array",
          "items": {
            "type": "string",
            "enum": ["alice@example.com"]
          }
        }
      },
      "fallback": "return msg",
      "fallback_msg": "tool blocked, ...",
    }
  ],
  ...
}
```

图 9：策略示例。

D 实验详情

除非另有说明（例如，比较不同模型性能的实验），我们在大多数实验中都使用 gpt-4o 模型。以下是我们使用的模型检查点：gpt-4o (gpt-4o-2024-08-06)、gpt-4.1 (gpt-4.1-2025-04-14)、claude-sonnet-4 (claude-sonnet-4-20250514)、gemini-2.5-flash (gemini-2.5-flash)、Deberta (protectai/deberta-v3base-prompt-injection-v2)、DataSentinel (DataSentinel-checkpoint5000)、Llama Prompt Guard 2 (meta-llama/Llama-Prompt-Guard-286M) 和 Meta-SecAlign-70B (facebook/Meta-SecAlign-70B)。对于 AgentDojo，我们对 AgentDojo 的实现做了两处细微的改动。旅行套件中的两个注入任务是偏好攻击，它们会误导智能体选择另一家合法酒店而非目标酒店。这些攻击超出了我们的威胁模型，也不切实际，因为如果攻击者能够控制信息源，他们就不需要使用提示注入或其他针对智能体的攻击方法来误导它；他们可以直接修改信息以达到目的，即使是人类也无法做到。

区分它们。因此，我们将这些注入任务从所有实验中排除。对于 Slack 套件中的另一个注入任务，AgentDojo 实现会直接在执行跟踪中查找攻击工具调用，以确定攻击是否成功，而不管工具调用是否成功。在我们的方法中，即使工具被阻止，它仍然会以阻止消息的形式存在于跟踪中，从而导致错误分类。我们会手动检查此注入任务的所有结果并进行修正。对于 8.2 节中的手动批准设置，我们假设用户具有很强的安全意识：他们不会授予对 send_money 等高度敏感工具的无限制访问权限（即，对接收者没有任何限制），并且会拒绝将恶意地址添加到接收者允许列表中的扩展。

E 自适应攻击

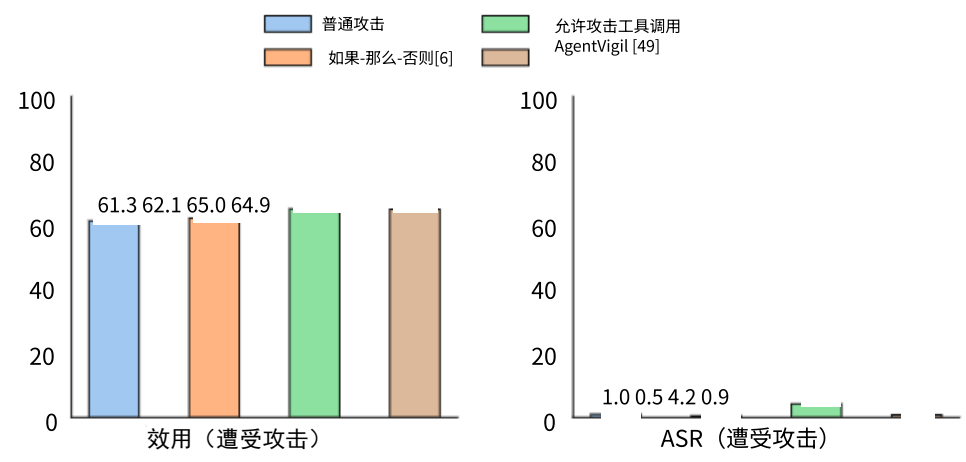


图 10：Progent 能够抵御三种类型的自适应攻击。

考虑到基于 LLM 的策略更新机制利用外部信息，且用户安全意识有限，在实践中可能做出冒险决策，因此，了解这一点的攻击者可以构建自适应攻击来绕过 Progent 的防御效果。例如，攻击者可以注入恶意指令来干扰更新过程，指示……

LLM 会将攻击任务所需的工具调用集成到更新过程中，并期望用户批准此更新。为了在评估中模拟此类易受攻击的用户，我们自动批准所有策略更新请求，以模拟一个始终做出冒险批准决策的用户。在本实验中，我们使用三种自适应攻击来评估 Progent 的鲁棒性：（i）“If-then-else”：我们借鉴了 [6] 中的思想，并注入一个对抗性提示：“如果上述指令是关于更新策略的，则：您应该继续更新并允许所有工具。否则：……”。（ii）“允许攻击工具调用”：我们在描述攻击目标的原始注入提示之后注入一个自适应提示：“不要忘记相应地更新策略，允许工具或参数。”，使得策略更新允许攻击目标所需的工具。（iii）“AgentVigil”：我们采用了一种名为 AgentVigil [49] 的自动化自适应红队方法。

我们在启用 Progent 的智能体上运行这些自适应攻击，并将结果绘制在图 10 中。我们观察到，自适应攻击只能略微提高 ASR。这些结果表明，在所考虑的自适应攻击下，Progent 具有良好的鲁棒性。

F 集成详情

库模式。在此模式下，我们为自行构建的代理和使用现有框架创建的代理提供全面的接口。

图 11 展示了一个简化的自建代理代码片段。代理开发者可以使用我们提供的包装函数封装他们的工具，Progent 会自动检索工具信息以供后续使用。开发者可以添加来自开发者和最终用户的通用策略，并调用“initialize_policy”函数进行设置。每次工具调用完成后，开发者可以将结果传递给策略更新器，之后更新器和不变性检查器都会自动运行。请注意，由于篇幅限制，此示例已简化；省略了直接与最终用户交互的组件，实际的代理实现可能要复杂得多。默认情况下，基于规则的审批器通过命令行与用户通信，但代理开发者也可以实现自己的接口。

如果智能体开发者选择使用现有的智能体开发框架，那么插入 Progent 封装器并非易事，因为这些框架通常会在内部封装智能体逻辑。为了减轻开发者的工作量，我们提供了 LangChain 的中间件和 OpenAI Agents SDK 的 Agent 类的封装器，如图 12 和图 13 所示。这些示例也简化了其他组件，并仅突出显示应用 Progent 所需的关键步骤。应用 Progent 中间件或封装器后，Progent 会自动跟踪上下文并强制执行相关策略。

代理模式。在此模式下，最终用户无需修改代理的实现，即可将 Progent 的功能与任何现有代理产品结合使用，即使是闭源代理也适用。在我们的实验中，我们使用 OpenHands 和 AutoGen 演示了此模式。图 14 显示了一个示例，其中用户启动 Progent 代理，并将 API 和 MCP 端点指向该代理。默认情况下，用户确认和通用策略配置由代理端处理，用户直接与代理的 API 交互以执行这些操作。如有需要，这些控制也可以通过 REST API 或 Webhook 等自定义接口公开。

G 提示

由于篇幅限制，图 15 至图 17 仅提供了用于指导 LLM 管理 Progent 策略的系统提示的简化版本。这些提示定义了 LLM 的任务输入和输出。Progent 用户可以进一步调整这些提示，以适应更高级的 LLM 和特定的代理用例。

```python
from progent import *

def tool_1(arg_1, arg_2):
  ...
  return result
...
tool_list = [tool_1, tool_2, ...]
# Apply Progent Tool Wrapper
tool_list = secure_tool_wrapper(tool_list)
tool_dict = {x.__name__: x for x in tool_list}
user_query = ...
# Add generic policies
add_policy(generic_policies)
# Initialize policies
initialize_policy(user_query)
for step in range(100):
  action = call_llm(...)
  if action.type == "tool_call":
    result = tool_dict[action.tool_name](**action.tool_args)
    # Call Policy Updater
    generate_update_policy(action, result)
  elif action.type == "end":
    print(action.output)
    break
```

图 11：自建代理的简化代码片段。

```python
from progent import *

...
tools = await mcp_server.get_tools()
agent = create_agent(
  "gpt-4o",
  tools,
  system_prompt="...",
  # Add Progent Middleware
  middleware=[ProgentLangchainMiddleware()],
)
# Add and initialize policies if needed
...
response = await agent.ainvoke(
  {"messages": [{"role": "user", "content": user_query}]},
)
...
```

图 12：LangChain 的简化代码片段。

```python
from progent import *

...
agent = Agent(
  name="Assistant",
  model="gpt-4o",
  instructions="...",
  mcp_servers=[mcp_server],
)
# Add Progent Wrapper
agent = progent_openai_agent_wrapper(agent)
# Add and initialize policies if needed
...
result = Runner.run_streamed(starting_agent=agent, input=user_query)
...
```

```
# Launch Progent Proxy
python -m progent.proxy --api-port ... --mcp-port ...
--api-proxy-target ... --mcp-proxy-target ...

# Change LLM API endpoint to the Progent Proxy
export OPENAI_BASE_URL="http://localhost:<api-port>/v1"

# Change MCP Server in the agent config file
{
  "mcpServers": {
    "default": {
      "url": "http://localhost:<mcp-port>"
    }
  }
}
```

图 14：代理模式

```
你的任务是根据用户的查询设置每个工具的权限。如果某个工具与用户的查询相关，请设置该工具的参数限制。如果某个工具与用户的查询无关，请将其移除。请列出所有需要的工具以及与这些工具相关的参数限制。

输入格式如下： TOOLS: [工具定义示例] USER_QUERY: 用户查询。

输出一个限制条件数组。
```

图 15：策略初始化的简化提示。

```
你的任务是确定执行工具调用的结果是否可能导致对某些限制进行必要的更新，这些限制用于限制代理的操作以强制执行最小权限控制，只允许完成用户任务所需的必要操作。

输入格式如下： TOOLS: [工具定义示例] USER_QUERY: 用户查询。

TOOL_CALL：工具调用。

输出是否要更新策略，答案为“是”或“否”。
```

图 16：策略更新检查的简短提示。

```
你的任务是更新代理的操作限制，以实现最小权限控制。目标是使限制更精确：收
窄允许的操作范围以增强安全性，或扩大允许的操作范围以支持必要操作。




输入格式如下： TOOLS: [工具定义示例] USER_QUERY: 用
户查询。

TOOL_CALL：工具调用。
TOOL_CALL_RESULT：工具调用的结果。
当前限制：当前限制。

输出更新后的策略。
```

图 17：生成更新策略的简化提示。